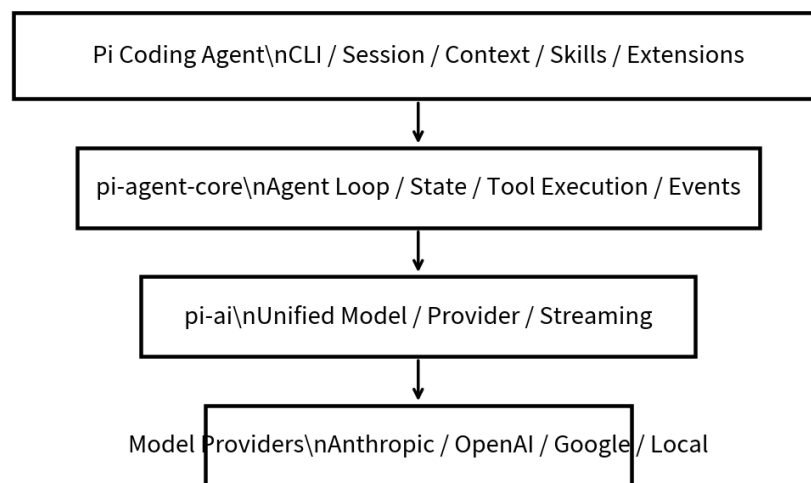


Pi Agent Cookbook

从最小 Coding Harness 到可扩展 Agent Runtime 的工程教程

安装 • Context Engineering • Skills • Extensions • Session • Compaction • SDK • Security

Pi 的四层架构：把 Provider、Runtime、Harness、UI/Domain 分离



文档基线：Pi Latest 文档，核对日期 2026-08-13

面向：Coding Agent / Agent Runtime / Context Engineering 实践者

如何使用这本 Cookbook

这不是一份“命令大全”。它把 Pi 当作一个可以拆开观察的 Agent Harness：先理解最小循环，再逐步添加上下文、流程知识、扩展、状态、安全边界和程序化集成。每个 Recipe 都尽量同时回答两个问题：为什么要这样设计，以及如何落地。

版本提醒 旧文章常见 `badlogic/pi-mono` 与 `@mariozechner/pi-coding-agent`；当前官方仓库与 npm 包已经使用 `earendil-works/pi` 与 `@earendil-works/*`。本书的命令、目录和 API 以 2026-08-13 的 Latest 文档为基线。

项目	说明
阅读顺序	首次使用：Part I -> Part II -> Part III；自研 Runtime：Part I -> Part V -> Part VI -> Part VIII。
Recipe 结构	目标 -> Know-why -> Know-how -> 实操 -> 验证 -> 常见坑 -> 演进。
安全原则	Pi 默认以当前用户权限运行。Project Trust 不是 sandbox。面向不可信仓库或无人值守任务时，应额外使用容器/VM/策略边界。
建议实验方式	用一个可丢弃的 Git 仓库做实验；每个 Recipe 单独提交，便于对比 Agent 行为。

目录

Part I 心智模型与架构

1. Pi 是什么：最小 Harness 的设计哲学
2. 四层架构：Provider / Runtime / Harness / UI
3. Agent Loop：最小循环为什么足够重要
4. Tool Design：为什么默认只需要少量通用工具
5. Context Engineering：上下文不是聊天记录

Part II 从零开始

6. 安装、认证与第一个 Session
7. 日常交互：文件、Shell、模型、消息队列
8. Provider、Model 与本地模型
9. 项目初始化与目录约定

Part III Customization Cookbook

10. AGENTS.md：项目长期约束
11. SYSTEM.md / APPEND_SYSTEM.md
12. Prompt Templates：重复任务入口
13. Skills：沉淀程序化 know-how
14. Extensions：机制与策略分离
15. Recipe：注册一个自定义 Tool
16. Recipe：路径保护与权限 Gate
17. Recipe：事件、命令与状态
18. Pi Packages：分发团队能力

Part IV Session、Memory 与 Context 生命周期

19. Session Tree 与 Fork/Clone
20. Compaction：历史如何变成工作记忆
21. Branch Summarization
22. Steer 与 Follow-up：Human-in-the-loop
23. Plan-as-File 与可观察状态

Part V SDK 与自研 Agent

24. Agent Core 与 Event Model
25. SDK：最小嵌入示例
26. SDK：自定义 Tool 与 transformContext
27. JSON / RPC 模式
28. 从 Pi 反推一个 Agent Runtime MVP

Part VI Security 与 Production Hardening

29. Project Trust 的真实边界
30. Sandbox / Containerization

- 31. Read-only / Tool Allowlist
- 32. 可观测性、成本与 Eval
- 33. Failure Recovery 与 checkpoint

Part VII Advanced Patterns

- 34. Subagent：为什么不必写进 Core
- 35. MCP vs CLI vs Skill
- 36. Research / Review / Coding 三类工作流
- 37. Pi vs Claude Code vs OpenCode：选型视角

Part VIII 源码学习与能力沉淀

- 38. 源码阅读顺序
- 39. 8 周实验路线
- 40. 从会话到 Skill 的 know-how 累积机制

Appendix

- A. CLI 速查
- B. 推荐项目模板
- C. Troubleshooting
- D. 设计检查清单
- E. 参考资料

Part I 心智模型与架构

1. Pi 是什么：最小 Harness 的设计哲学

Pi 是一个面向终端的 Coding Agent Harness，同时又把底层模型访问、Agent Runtime、TUI 和 Coding Agent 分成独立包。它的核心价值不是“替你决定最佳工作流”，而是提供较小、可观察、可替换的机制，让工作流通过 Context Files、Skills、Prompt Templates、Extensions 与 Packages 逐步长出来。

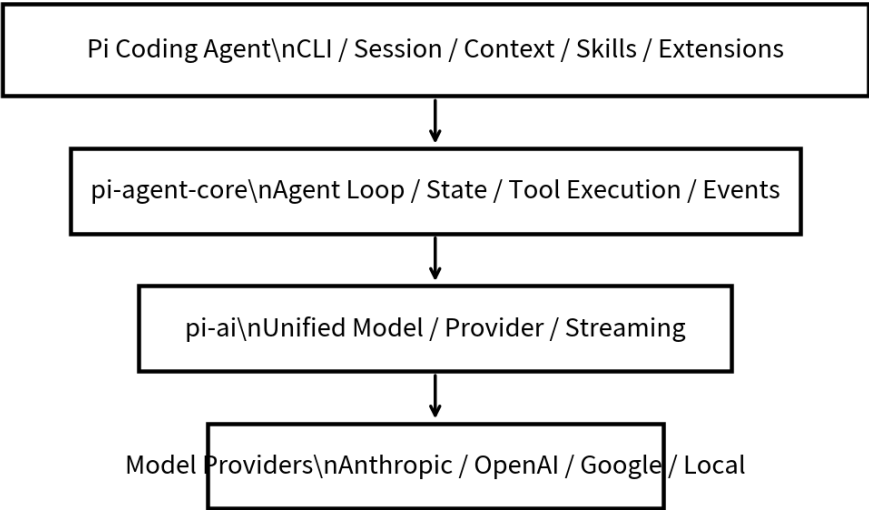
Know-why 很多 Agent 框架把 Planner、Todo、Subagent、MCP、Permission、Memory 一开始就做成内建状态机。这样开箱即用，但也会把模型行为变成 Harness 的隐式副作用。Pi 反向选择：Core 尽量小，把 workflow-specific behavior 外移。

Know-how 学习 Pi 时不要先安装大量第三方扩展。先用默认 read/write/edit/bash 做 3-5 个真实任务，记录哪些失败确实来自“缺机制”，再决定是否引入 Skill、Extension、Sandbox 或 Subagent。

- Core 小：便于理解模型每轮到底看到了什么。
- 机制可扩展：需要 plan mode、permission、subagent 时通过 extension/package 增加。
- Context 可控：长期规则、任务入口、按需能力分别放在不同层。
- Session 可回溯：历史不是只能线性继续，而是可以 tree/fork/clone。

2. 四层架构：Provider / Runtime / Harness / UI

Pi 的四层架构：把 Provider、Runtime、Harness、UI/Domain 分离



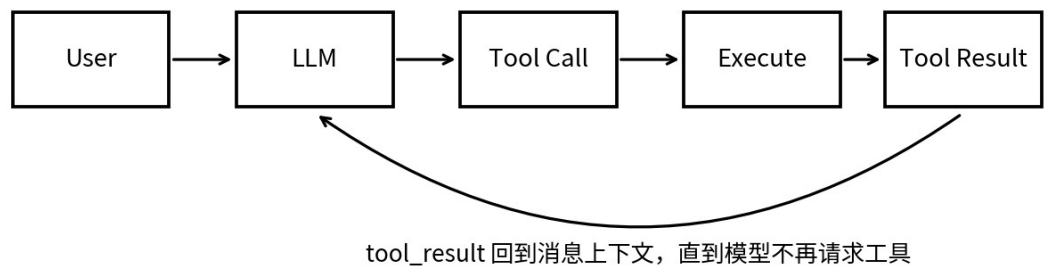
项目	说明
@earendil-works/pi-ai	多 Provider 的模型与流式 API。真正困难的是跨

	Provider 的 message、tool call、reasoning、usage 与上下文连续性。
@earendil-works/pi-agent-core	Stateful agent、Tool execution、事件流、消息转换。它是最适合研究 Agent Runtime 的层。
@earendil-works/pi-coding-agent	把 Runtime 变成 Coding Harness：Session、Context Files、Skills、Extensions、CLI/SDK。
@earendil-works/pi-tui	终端交互与渲染。应被视为 UI，而不是 Agent 的核心逻辑。

可迁移原则 自研 Agent 时优先形成 Provider Adapter -> Agent Runtime -> Domain Harness -> UI/Integration 的依赖方向。不要让 UI 直接拥有消息循环、工具执行和持久化规则。

3. Agent Loop：最小循环为什么足够重要

最小 Agent Loop：Reason -> Act -> Observe -> Reason



最小 Agent 的“秘密”仍然是：模型产生 tool call，宿主执行工具，把 tool result 加回上下文，再调用模型，直到模型停止请求工具。真正的复杂性通常不是循环本身，而是围绕循环的上下文转换、工具策略、事件、恢复、持久化和安全边界。

```
messages.append(user_message)
while True:
    response = llm(messages, tools)
    messages.append(response)

    if not response.tool_calls:
        break

    results = execute(response.tool_calls)
    messages.extend(results)
```

Know-why 如果连最小 Loop 都没有单独建模，后面的 Planner、Memory、Subagent 往往会和 UI、工具调度缠在一起，最终无法单测，也无法解释失败来自模型还是 Runtime。

4. Tool Design：为什么默认只需要少量通用工具

Pi 默认首次会话给模型 read、write、edit、bash 四个工具；grep、find、ls 也作为内建只读工具可通过 tool 选项启用。这个选择把“工具表面积”压得很小，让 Shell 承担大量长尾能力。

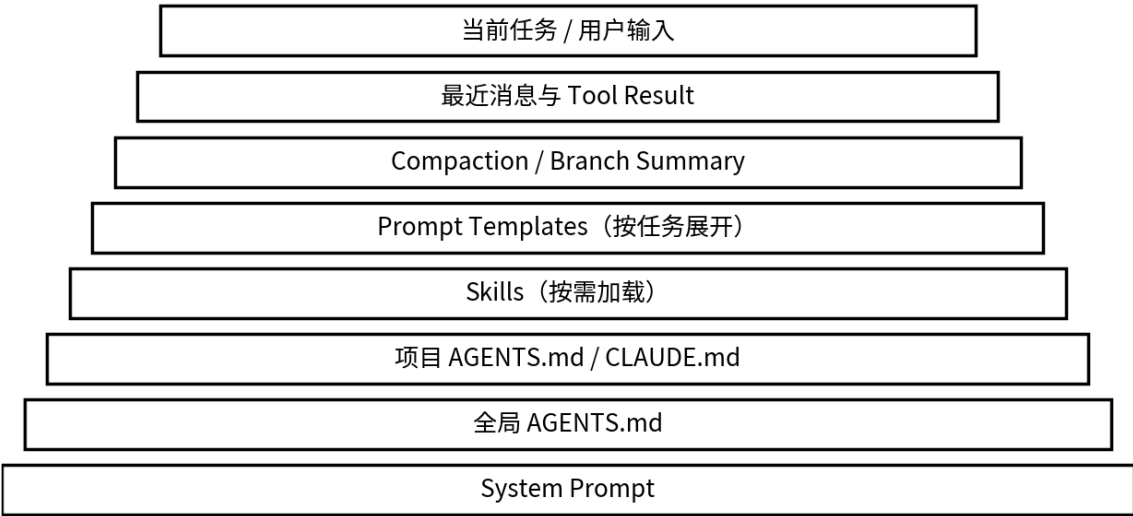
项目	说明
read	把文件/图片内容变成模型可用的 observation。
write	创建或整体覆盖文件。适合新文件或明确重写。
edit	局部替换，适合最小 diff。
bash	统一访问 git、测试、构建、搜索、格式化、脚本等 CLI。

Know-why Tool Schema 也消耗上下文，并且工具越相似，选择歧义越大。Coding Model 已经熟悉 shell，因此通用 primitive 可以减少 schema 数量和 Harness 维护成本。

边界 “少量通用工具”适合受控开发环境；面向生产业务 API、高风险写操作或强审计场景时，typed domain tool + deterministic policy 往往更合适。

5. Context Engineering：上下文不是聊天记录

Pi Context Stack：Context Window 是工作内存，不是知识仓库



工程目标：减少长期常驻内容；把流程知识做成按需加载；把历史压缩成未来仍有用的信息。

把 Context Window 当成每一轮推理的“有限工作内存”。它不应长期承载全部历史、全部工具、全部流程文档。不同信息有不同生命周期：长期规则常驻，任务模板按调用展开，技能按需加载，旧历史压缩，废弃分支只保留对未来仍有价值的摘要。

设计公式 $\text{Context}(t) = \text{System} + \text{稳定项目规则} + \text{当前可用能力描述} + \text{最近消息/结果} + \text{压缩摘要} + \text{当前任务}$ 。优化目标不是“塞更多”，而是提高每个 token 对当前决策的边际价值。

Part II 从零开始

6. 安装、认证与第一个 Session

```
# 推荐安装
npm install -g --ignore-scripts @earendil-works/pi-coding-agent

# 进入一个 Git 项目
cd /path/to/project
pi
```

进入交互界面后，可用 /login 选择订阅或 API-key Provider；也可在启动前通过环境变量提供 API Key。建议第一轮让 Pi 做“只理解、不修改”的任务，确认 cwd、模型、上下文文件和工具加载情况。

- 1. 在一个可丢弃或已提交干净状态的 Git 仓库中启动 pi。
- 2. 执行 /login 或配置 Provider API key。
- 3. 输入：Summarize this repository and tell me how to run its checks.
- 4. 查看它如何使用 read/bash，并确认结论是否和仓库脚本一致。
- 5. 执行 git status，确认第一次实验没有非预期改动。

验证 能完成仓库结构总结、找到测试/检查命令，并且你能解释它为什么调用了每个工具，才算完成第一阶段。

7. 日常交互：文件、Shell、模型、消息队列

项目	说明
@ 文件引用	在编辑器中输入 @ 模糊搜索文件；也可在 CLI 直接使用 @README.md。
!command	运行 shell，并把输出送入模型上下文。
!!command	运行 shell，但不把输出送入模型；用于纯人工查看。
/model	交互式切换模型。
Enter during run	排入 steering message，当前 assistant turn 的 tool calls 完成后交付。
Alt+Enter during run	排入 follow-up，等 Agent 当前工作全部结束后再交付。
Escape	中止，并把排队消息恢复到编辑器。

Know-why Steer 和 Follow-up 是两种不同的人机协作语义：前者改变正在进行的方向，后者是排队的下一项工作。把两者都实现成“新 user message”会造成执行时机不确定。

8. Provider、Model 与本地模型

Pi 把 Provider 与 Model 视为可替换层。日常使用可通过 `/model`、`--provider`、`--model` 切换；自定义 OpenAI-compatible 模型则通过 `~/pi/agent/models.json` 描述。

```
# 查看可用模型
pi --list-models

# 指定 provider/model
pi --provider openai --model <model-id>

# 只允许一组模型参与快捷切换
pi --models "pattern-a,pattern-b"
```

本地模型的关键不是“名称”，而是协议兼容能力：是否支持 tool calling、developer/system role、reasoning 参数、图片、并发等。配置时应按 capability 建模，而不是只假设“OpenAI-compatible 就完全一致”。

```
{
  "providers": {
    "local": {
      "baseUrl": "http://localhost:11434/v1",
      "api": "openai-completions",
      "apiKey": "local",
      "models": [
        { "id": "your-local-model" }
      ]
    }
  }
}
```

实践建议 先用云端强模型验证 Harness 和 workflow，再替换本地模型。否则“模型能力不足”和“Runtime/Context 设计错误”会同时出现，很难定位。

9. 项目初始化与目录约定

```
my-project/
├── AGENTS.md
├── src/
├── tests/
├── .pi/
│   ├── settings.json
│   ├── prompts/
│   ├── skills/
│   └── extensions/
```

分层原则 AGENTS.md 放长期稳定规则；Prompt Template 放重复任务入口；Skill 放按需流程知识与资源；Extension 放需要真正执行或拦截的机制。

Part III Customization Cookbook

10. AGENTS.md：项目长期约束

Pi 启动时可以从全局、父目录与当前目录加载 AGENTS.md 或 CLAUDE.md；如果某目录存在 AGENTS.override.md，会在该目录层覆盖普通上下文文件。适合放“每次任务都应该知道”的稳定事实。

```
# Project Instructions

## Architecture
- Domain code lives in src/domain.
- Infrastructure may depend on domain; domain must not depend on infrastructure.

## Verification
After code changes:
1. npm run typecheck
2. npm test

## Editing
- Prefer minimal diffs.
- Do not reformat unrelated files.

## Safety
- Never run production migrations from this repository.
```

反模式 把一次性任务、很长的教程、所有工具说明、所有历史经验都放进 AGENTS.md，会让每一轮都支付上下文成本。稳定规则越短、越可验证，效果越好。

- 写“可观察行为”，少写空泛人格。
- 把验证命令写明确，让 Agent 能执行自检。
- 写清依赖方向、禁止边界和变更范围。
- 定期删除已经由代码/CI 自动保证的冗余规则。

11. SYSTEM.md / APPEND_SYSTEM.md

项目或全局可以用 SYSTEM.md 替换默认 system prompt；APPEND_SYSTEM.md 则只追加。替换是强动作：一旦覆盖默认 prompt，你也同时承担工具使用约束、行为说明和基本交互契约的维护成本。

何时使用 只有当你在做“自定义 Harness 行为实验”或需要完全控制 system 层时再用 SYSTEM.md。日常项目约束优先 AGENTS.md；新增规则优先 APPEND_SYSTEM.md。

12. Prompt Templates：重复任务入口

Prompt Template 是可复用 Markdown Prompt，可通过 /name 展开。它适合“每次开始任务时都需要一套结构化要求”，但不应该一直常驻上下文。

```
# .pi/prompts/review.md
---
description: Review current git changes
```

```
---

Review `git diff`.

Focus on:
1. correctness
2. regressions
3. error handling
4. security
5. missing tests

Rank findings by severity.
```

Know-why 把重复任务入口做成模板，可以减少“每次手写 prompt 的漂移”，同时不让这套说明长期占用所有任务的上下文。

13. Skills：沉淀程序化 know-how

Skill 是按需加载的能力包，可以包含 SKILL.md、脚本、参考资料和资产。它适合沉淀“如何完成某类任务”，而不是“这个项目永久是什么样”。

```
# .pi/skills/code-investigation/SKILL.md
---
name: code-investigation
description: Investigate an unfamiliar subsystem and produce an evidence-backed architecture note.
---

# Workflow
1. Locate entry points.
2. Build a small call graph.
3. Identify state transitions and side effects.
4. Find the smallest relevant tests.
5. Validate assumptions by running targeted checks.
6. Produce: architecture, invariants, risks, open questions.
```

Know-why Skill 是 procedural memory：它把“多次协作中反复有效的方法”变成可版本化资源。与巨大 system prompt 相比，它实现 progressive disclosure：需要时才加载。

安全 Skill 可以指导模型执行任意动作，也可能携带脚本。第三方 Skill 必须像代码依赖一样审查；项目级 Skill 只有在信任项目后加载。

14. Extensions：机制与策略分离

Extension 是 Pi 最强的扩展点：TypeScript 模块可以注册 Tool、Command、事件监听、UI、Provider，并对工具调用、Session、Context 等生命周期做干预。

项目	说明
----	----

Tool	给模型新增真正可执行的 capability。
Command	给人类新增 slash command。
Event	在 startup/session/agent/tool/input 等生命周期插入逻辑。
State	通过 session entry 或自有存储保存扩展状态。
UI	状态栏、对话框、组件、渲染器。
Policy	可在 tool event 上做 deterministic gate，而不是只靠 prompt。

核心原则 Mechanism in Core, Policy in Extension。Runtime 提供事件与控制点；“是否允许、如何规划、是否 spawn 子 Agent” 由业务策略决定。

15. Recipe：注册一个自定义 Tool

目标：增加一个“读取项目健康信息”的工具，但保持输出短、结构稳定、对模型有决策价值。下面代码是示意骨架，实际类型/API 以当前 extensions 文档为准。

```
// .pi/extensions/project-health.ts
export default function (pi) {
  pi.registerTool({
    name: "project_health",
    description: "Return concise project health signals: git state and available check scripts.",
    parameters: { type: "object", properties: {} },
    async execute(_args, ctx) {
      const git = await pi.exec("git", ["status", "--short"]);
      const pkg = await pi.exec("node", ["-e",
        "const p=require('./package.json'); console.log(JSON.stringify(p.scripts||{}))"
      ]);
      return {
        content: [{
          type: "text",
          text: `git_changes=${git.stdout.trim()} || "clean"\nscripts=${pkg.stdout.trim()}`
        }]
      };
    }
  });
}
```

Tool 设计检查 Description 要回答“什么时候用”；参数要减少歧义；结果只返回下一步决策需要的信息；大量原始数据应在工具内部过滤/聚合。

16. Recipe：路径保护与权限 Gate

目标：把“不要修改敏感配置”从自然语言建议升级成 Runtime 级硬约束。正确思路是拦截 write/edit/bash 等 tool call，先做确定性判断，再决定 block、ask 或 allow。

```
// 伪代码：具体事件字段请以当前 extension API 为准
pi.on("tool_call", async (event, ctx) => {
  const target = extractTargetPath(event);

  if (target && isProtected(target)) {
    return {
      block: true,
      reason: "Protected path: change requires an explicit human-controlled workflow."
    };
  }
});
```

项目	说明
Gate 1 - hard deny	永远禁止的路径/动作。
Gate 2 - rule match	根据 cwd、工具、参数、文件类型判定风险。
Gate 3 - approval	只有 UI/业务允许时才进入人工确认。
Gate 4 - execute	执行并记录 audit event。

Know-why Prompt 规则是概率性的。真正的权限控制必须在模型输出之后、工具执行之前，以 deterministic code 实现。

17. Recipe：事件、命令与状态

一个稳定 Extension 通常不是“写一个 tool 就结束”，而是围绕生命周期形成小型状态机。建议把事件处理器保持短小，让状态变化可持久化、可追踪。

```
export default function (pi) {
  pi.registerCommand("checkpoint", {
    description: "Create a named working checkpoint",
    async handler(args, ctx) {
      const name = args.trim() || "checkpoint";
      // 1) inspect git state
      // 2) store a session entry / label
      // 3) optionally create an external checkpoint
      pi.setSessionName(name);
    }
  });

  pi.on("agent_end", async (event, ctx) => {
    // emit metrics / validation summary / cleanup
  });
}
```

状态原则 对 Agent 和人都需要查看、修改、版本化的状态，优先落到普通文件或 Session entry；只对 Extension 内部有意义的 ephemeral state 才留在内存。

18. Pi Packages：分发团队能力

Package 可以把 extensions、skills、prompts、themes 一起分发。团队化之后，真正重要的是版本、审查、默认启用范围和升级策略，而不是“能安装就行”。

```
{
  "name": "company-pi-kit",
  "version": "1.0.0",
  "keywords": ["pi-package"],
  "pi": {
    "extensions": ["/extensions"],
    "skills": ["/skills"],
    "prompts": ["/prompts"]
  }
}
```

```
# 安装 npm / git package
pi install npm:@your-org/company-pi-kit
pi install git:github.com/your-org/company-pi-kit

# 项目级安装可使用 -l
pi install -l npm:@your-org/company-pi-kit
```

安全 Pi Package 中 Extension 是可执行代码，Skill 也能引导模型执行程序。第三方 package 应纳入供应链审查与版本锁定。

Part IV Session、Memory 与 Context 生命周期

19. Session Tree 与 Fork/Clone

Pi 自动保存 Session，并把 Session 内历史视为树，而不是只能向前追加的链。/tree 可以跳到历史任意点继续，/fork 从早期 user message 生成新 Session，/clone 把当前 active branch 复制到新 Session。

Know-why Coding 是搜索过程：方案 A 失败后返回分叉点尝试 B 很常见。线性聊天会把错误路径继续留在工作上下文；树结构允许“回到决定发生前”。

6. 给当前任务 /name，方便之后搜索。
7. 实现一个方案并运行验证。
8. 若发现核心假设错误，用 /tree 跳回假设之前。
9. 需要保留旧会话证据时用 /fork 或 /clone，而不是简单清空。
10. 新分支开工前写一句“为什么放弃旧方案”，降低未来认知负担。

20. Compaction：历史如何变成工作记忆

当上下文接近模型窗口时，Pi 可以自动 compaction，也可手动 /compact [prompt]。目标是总结较旧内容、保留近期工作，从而释放上下文。

```
/compact
# 带自定义压缩重点
/compact Focus on architecture decisions, files changed, failing tests, and unresolved risks.
```

压缩时要保什么 决策、证据、关键文件、当前 invariants、失败原因、尚未解决的问题、已执行验证。不要把逐句聊天和重复 tool output 当作必须保留的信息。

21. Branch Summarization

当从树中离开一个已经深入探索的分支时，“全部丢弃”会损失发现，“全部携带”会污染新方向。更合理的是把 abandoned branch 变成结构化摘要：只保留对未来仍有价值的知识。

```
Abandoned branch summary
- Hypothesis tested: ...
- Evidence found: ...
- Why abandoned: ...
- Files / symbols worth remembering: ...
- Tests / commands already run: ...
- Reusable insight: ...
```

Memory 原则 Memory 的单位不应该是“消息”，而应该是“对未来决策仍有价值的信息”。这是从 Chat

History 走向 Agent Working Memory 的关键。

22. Steer 与 Follow-up：Human-in-the-loop

在长任务中，用户可能在 Agent 工作时发现方向错误。Steer 让新消息在当前 assistant turn 的 tool calls 完成后尽快生效；Follow-up 则明确表示“当前工作完成后再做”。

项目	说明
Steer	改变正在进行的任务方向。例如：不要再改接口，先定位测试失败根因。
Follow-up	排队下一项独立任务。例如：完成修复后再补一份设计说明。
Abort	真正停止当前执行，而不是一种普通反馈。

Runtime 设计 自研 Agent 不要只有 prompt() 和 cancel() 两个动作。至少把 steering queue、follow-up queue、abort 三种控制面分开建模。

23. Plan-as-File 与可观察状态

Pi 不强制内建 Plan Mode。一个很实用的替代是把计划写到 PLAN.md：模型可读、人可改、Git 可 diff、其他工具也能处理。

```
# PLAN.md

## Goal
...

## Constraints
...

## Steps
- [ ] Reproduce failure
- [ ] Identify invariant
- [ ] Implement minimal fix
- [ ] Run focused tests
- [ ] Run regression checks

## Evidence / Decisions
...
```

什么时候不要这么做 如果计划只是模型内部几步即时推理，强迫落盘会制造仪式成本。只有需要跨轮次、跨人、跨 Agent 或可审计时，文件状态才真正有价值。

Part V SDK 与自研 Agent

24. Agent Core 与 Event Model

Agent Core 的关键不是一个返回字符串的 `run()`，而是 `stateful agent + event stream`。事件让 UI、日志、Eval、Telemetry、Permission 与持久化从 Agent Loop 解耦。

```
agent_start
  turn_start
    message_start
    message_update ...
    tool_execution ...
    message_end
  turn_end
agent_end
```

Know-why 如果 Runtime 只暴露 `final answer`，外部系统无法可靠知道模型正在思考、工具是否开始、工具是否失败、用户何时 `steer`，也无法做细粒度成本和延迟分析。

25. SDK：最小嵌入示例

Coding Agent SDK 可以创建 `AgentSession`，并让你选择内存 Session 或持久化后端。下面示例展示最小生命周期：创建 runtime、session、订阅事件、发送 prompt。

```
import {
  createAgentSession,
  ModelRuntime,
  SessionManager
} from "@earendil-works/pi-coding-agent";

const modelRuntime = await ModelRuntime.create();
const { session } = await createAgentSession({
  sessionManager: SessionManager.inMemory(),
  modelRuntime,
});

session.subscribe((event) => {
  if (event.type === "message_update" &&
    event.assistantMessageEvent?.type === "text_delta") {
    process.stdout.write(event.assistantMessageEvent.delta);
  }
});

await session.prompt("Explain this repository architecture.");
```

实验 先只做“无 UI、单 Session、只读工具”的最小程序。确认事件顺序与 state 变化，再加入自定义工具和持久化。不要一开始就同时接 WebSocket、数据库和多 Agent。

26. SDK：自定义 Tool 与 transformContext

Agent Runtime 的两个高杠杆扩展点分别是：Tool 决定模型能作用于什么环境；Context transform 决定模型每轮真正看到什么。后者尤其适合注入外部状态、裁剪 UI-only message、做自定义 memory。

```
AgentMessage[]
-> transformContext()      # prune / inject / summarize
-> AgentMessage[]
-> convertToLlm()          # filter UI-only/custom messages
-> LLM Message[]
-> Model
```

Know-how 把“应用消息”和“LLM 消息”分开。UI 事件、审批状态、内部 trace 不一定应该直接变成模型消息；通过 convert/transform 层显式决定。

27. JSON / RPC 模式

除了交互 TUI，Pi 还提供 print、JSON event stream 和 RPC 等模式。适合把 Pi 当作子进程集成，而不需要直接嵌入 SDK。

```
# one-shot
pi -p "Summarize this codebase"

# structured events
pi --mode json -p "Review the current diff"

# process integration
pi --mode rpc
```

选型 需要快速接现有进程：RPC/JSON；需要强类型、直接操作 Session/Agent state：SDK；需要给人使用：Interactive。

28. 从 Pi 反推一个 Agent Runtime MVP

如果你正在自己写 Agent Runtime，可以按下面顺序实现，而不是一开始就做 Multi-Agent Framework。

11. Model Adapter：统一 stream / tool call / usage / finish reason。
12. AgentState：messages、model、system prompt、tools、status。
13. Agent Loop：tool call -> execute -> result -> continue。
14. Event Bus：message/tool/turn/agent lifecycle。
15. Tool Registry：schema、执行器、超时、错误类型。
16. Context Transform：裁剪、注入、压缩、UI message 过滤。
17. Session Store：append-only entry + active branch。
18. Control Plane：steer、follow-up、abort。
19. Policy Hooks：before/after tool call。
20. 最后再加 Skill、Subagent、Planner、MCP。

成功标准 每增加一层前，写一个可复现的失败用例。如果没有失败用例证明它必要，就不要先做抽象。

Part VI Security 与 Production Hardening

29. Project Trust 的真实边界

Project Trust 控制是否加载项目本地 settings/resources/packages/extensions 等内容。它解决的是“仓库能否向 Pi 注入本地扩展与配置”的信任问题，不是操作系统权限边界，也不会在会话开始后限制模型通过工具做什么。

必须记住 Trust != Sandbox。信任一个项目以后，Extension 可能执行代码，Skill 可能引导模型执行动作；而默认工具仍然以启动 Pi 的用户权限运行。

30. Sandbox / Containerization

当仓库不可信、任务无人值守、需要网络/凭证隔离，应该把真实安全边界放到容器、VM 或策略化 sandbox 外层。Pi 官方文档也把 containerization 作为独立安全层讨论。

```
Host
├── Sandbox / Container / VM
│   ├── project checkout
│   ├── limited credentials
│   ├── network policy
│   └── pi
│       └── tools
```

- 文件系统：只挂载需要的工作目录。
- 凭证：按任务注入最小权限、短生命周期凭证。
- 网络：默认最小化，必要时显式 allowlist。
- 进程：限制可见 host 资源与特权。
- 审计：保留 tool event、命令、文件 diff 和 exit status。

31. Read-only / Tool Allowlist

分析、Code Review、架构调研并不需要写权限。Pi 可以用 --tools allowlist 或 --exclude-tools 限制内建和扩展工具。

```
# 只读审查
pi --tools read,grep,find,ls -p "Review the code for correctness and risks"

# 禁用所有内建工具，只保留显式 extension/custom tools
pi --no-builtin-tools

# 完全无工具
pi --no-tools
```

Know-why Capability 最小化比“提示模型不要写”更可靠。对只读任务，最好的 write permission 是根本不给 write tool。

32. 可观测性、成本与 Eval

要把 Agent 变成工程系统，至少需要记录：每轮模型、上下文 token、tool call、工具耗时、错误、steer 次数、compaction、最终验证结果。Pi 的事件模型使这些数据可以在 UI 之外独立收集。

项目	说明
Outcome	任务是否真正完成；测试/评测是否通过。
Trajectory	走了哪些工具路径；是否重复搜索/反复编辑。
Context	上下文峰值、compaction 次数、summary 大小。
Cost	输入/输出/缓存 token 与模型成本。
Latency	模型、工具、整体 wall clock。
Intervention	人类 steer/abort/approval 的次数和原因。

Eval 原则 不要只评最终答案。Coding Agent 的失败经常来自 trajectory：选错文件、读错证据、工具输出过大、未验证、反复回滚。

33. Failure Recovery 与 checkpoint

Agent 的恢复能力来自三个层次：Git/文件系统 checkpoint 负责世界状态；Session tree 负责对话/决策状态；Compaction/summary 负责上下文状态。三者不要混为一谈。

项目	说明
World State	文件、数据库、远端资源。用 Git、快照、事务、幂等接口保护。
Agent State	Session tree、active branch、queued messages。
Context State	最近消息、summary、已加载技能、系统指令。

Know-how 每个可逆步骤应能回答“如何检测失败、如何回滚、如何恢复上下文”。这比单纯加 Retry 更重要。

Part VII Advanced Patterns

34. Subagent：为什么不必写进 Core

Pi 故意不把某一种 Subagent topology 固化到 Core。因为“子 Agent”可能是阻塞、并行、后台、只读 scout、planner、reviewer，也可能共享或隔离上下文。不同模式的生命周期和风险完全不同。

项目	说明
Scout	只读、快速定位文件/符号，输出证据摘要。
Planner	读证据，不写代码，输出 implementation plan。
Worker	完整工具集，执行具体变更。
Reviewer	读 diff 与测试，寻找回归，再反馈给 worker。

设计建议 Runtime 只需要提供 spawn/session/tool primitives；拓扑放到 Extension/Workflow。这样可以替换 parallel/chained/hierarchical 策略，而不用改 Core。

35. MCP vs CLI vs Skill

项目	说明
CLI	已有成熟命令行，模型熟悉参数；用 bash 即可调用。最低 Harness 成本。
Skill	CLI 使用需要流程知识、组织规则或脚本；按需加载说明。
Typed Tool	需要稳定参数、结构化结果、权限控制、观测和强约束。
MCP	跨应用统一暴露远端/本地能力、需要标准化发现与调用；但要关注 tool schema 的上下文成本。

决策法 不要问“能不能做 MCP”，而问：能力是否已有 CLI？模型需要多少流程知识？是否需要结构化输入输出和权限边界？工具 schema 是否会长期占用上下文？

36. Research / Review / Coding 三类工作流

同一个 Harness 应通过 tool policy + context + skill 形成不同 profile，而不是复制三套 Agent。

项目	说明
Research	read/grep/find/ls + web/search extension；强调证据、引用、分支摘要。
Review	只读工具 + diff/test observation；强调 severity、

	reproduction、false positive 控制。
Coding	read/write/edit/bash + checkpoint；强调最小 diff、targeted test、回归验证。

```
# 示例：只读 reviewer profile
pi --tools read,grep,find,ls \
  --name "review" \
  "Review the current repository. Do not modify files."
```

37. Pi vs Claude Code vs OpenCode ：选型视角

项目	说明
Pi	更像 programmable harness：核心小、Context/Extension/Session 可控，适合学习与自定义。
Claude Code	更像 opinionated coding product：产品层能力丰富、默认 workflow 更完整。
OpenCode	开源多模型 Coding Agent 产品，生态与可配置性较强。

正确问题 不是“谁最聪明”，而是：你需要多少默认产品能力？需要多大 Harness 可见性？是否要把 Runtime 嵌入自己的产品？安全、Provider、扩展和团队分发由谁负责？

Part VIII 源码学习与能力沉淀

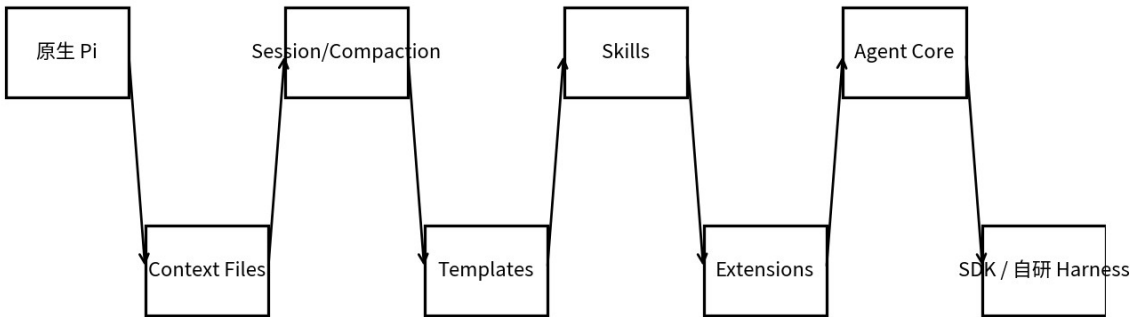
38. 源码阅读顺序

- 21. pi-ai: Model、Message、ToolCall、Stream、Provider abstraction。
- 22. pi-agent-core: Agent、AgentState、agent loop、event、tool execution。
- 23. coding-agent tools: read/write/edit/bash 的 schema 与 observation 设计。
- 24. Session Manager / session format: JSONL、tree、branch、entry。
- 25. Compaction: 触发条件、summary 输入、分支摘要。
- 26. Resource Loader: AGENTS、Skills、Prompt Templates、Extensions。
- 27. Extension System: hook、tool、command、UI、provider。
- 28. TUI: 最后再看渲染与快捷键。

Know-why 先读 TUI 很容易把“产品交互”误当成“Agent 架构”。正确顺序应该从数据模型和循环到 domain harness，再到 UI。

39. 8 周实验路线

学习路线：先观察最小 Harness，再逐步引入抽象



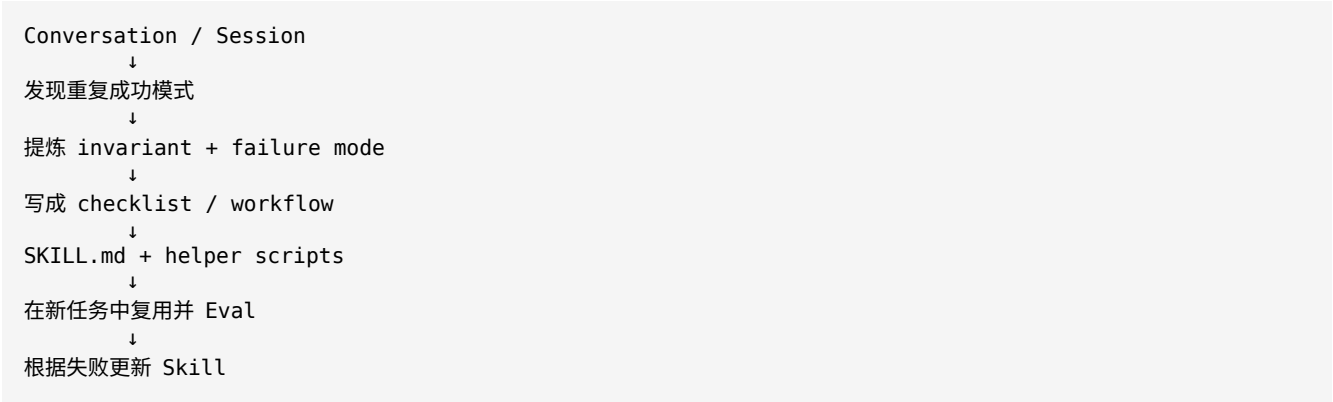
原则：每加一层机制，都要能回答“它解决了哪一个真实失败模式？”

项目	说明
Week 1	只用原生 Pi；记录 tool trajectory。
Week 2	设计 AGENTS.md；对比改前/改后行为。
Week 3	练习 /tree /fork /compact；观察 context 生命周期。
Week 4	做 3 个 Prompt Templates 和 1 个 Skill。
Week 5	写第一个 Extension：custom tool + command。

Week 6	写 deterministic permission/path guard，增加日志。
Week 7	使用 SDK 构造一个只读 Review Agent。
Week 8	重写一个最小 Agent Core，对照 Pi 的 state/event/context 设计。

40. 从会话到 Skill 的 know-how 累积机制

真正可累积的 Agent 能力，不是保存所有对话，而是不断把重复成功模式提炼成短、可验证、可版本化的程序知识。



项目	说明
应该沉淀	稳定步骤、输入/输出契约、判定规则、验证方法、常见失败、工具组合。
不该沉淀	偶然路径、一次性文件名、冗长聊天、无法验证的风格偏好、模型自我解释。
升级条件	同类任务至少重复出现；流程对不同项目仍成立；能明确判断成功/失败。

最终原则 Memory 的终点不是更长的上下文，而是更好的可重用能力。

Appendix A CLI 速查

项目	说明
pi	交互模式。
pi -p "..."	一次性 print 模式。
pi --mode json	JSONL 事件流。
pi --mode rpc	stdin/stdout RPC。
pi -c	继续最近 Session。
pi -r	浏览旧 Session。
pi --session <id>	打开指定 Session。
pi --no-session	不持久化。
/model	切换模型。
/settings	修改思考级别、主题、消息交付等。
/tree	Session tree 导航。
/fork	从旧 user message 新建 Session。
/clone	复制 active branch。
/compact	手动压缩上下文。
/reload	重载 extensions/skills/prompts/context 等。
/session	查看 Session 元信息。
/trust	保存项目 trust 决策。
/export	导出 Session。

```
# 文件参数
pi @README.md "Summarize this"
pi @src/app.ts @src/app.test.ts "Review together"

# 只读
pi --tools read,grep,find,ls -p "Review this repository"

# 精确加载一个 extension
pi --no-extensions -e ./my-extension.ts
```

Appendix B 推荐项目模板

```
project/
├── AGENTS.md
├── PLAN.md           # 只有长任务需要时创建
├── src/
└── tests/
```

```
└─ .pi/
   ├── settings.json
   ├── prompts/
   │   ├── investigate.md
   │   ├── review.md
   │   └── finish.md
   ├── skills/
   │   ├── code-investigation/
   │   │   └── SKILL.md
   │   ├── release/
   │   │   └── SKILL.md
   │   └── scripts/
   ├── extensions/
   │   ├── path-guard.ts
   │   └── checkpoint.ts
```

Appendix C Troubleshooting

项目	说明
Pi 没看到最新 AGENTS/Skill	执行 /reload；确认目录层级、Project Trust 与 discovery path。
模型反复调用相似工具	减少 tool schema；收紧 description；合并功能重叠的工具；缩短返回。
Context 很快爆满	检查大型 tool result、重复日志、常驻 instructions；使用 compact；把流程搬到 Skill。
本地模型工具调用差	先验证模型是否真正支持 tool calling；减少工具数量；使用更强模型作为对照组。
Agent 修改过多文件	AGENTS 明确 minimal diff；使用 edit；增加 path policy；每轮查看 git diff。
只做 Review 却发生写入	不要只靠 prompt，直接用 --tools read,grep,find,ls。
Extension 行为难调试	先减少到单事件/单 tool；记录 event 顺序和关键 state；避免在 handler 内做过多隐式工作。
切分支后旧错误知识污染	/tree 回到正确分叉点；必要时 fork；使用 branch summary 而不是继续线性对话。

Appendix D Agent 设计检查清单

- ☐ 能否画出最小 Agent Loop，并指出每一步的输入/输出？
- ☐ Provider、Runtime、Harness、UI 是否解耦？
- ☐ Tool schema 是否只暴露模型真正需要的能力？
- ☐ Tool result 是否做了过滤/聚合，而不是把原始数据全部塞入 Context？
- ☐ 长期规则、任务模板、Skill、Extension 是否各归其位？

- ☐ 是否支持 read-only profile?
- ☐ 权限是否在执行前由确定性代码控制，而不是只写 prompt?
- ☐ Project Trust 与 Sandbox 是否明确区分?
- ☐ Session 是否可分支、恢复和克隆?
- ☐ 旧历史是否可压缩？废弃分支是否能只保留有用摘要？
- ☐ 是否支持 steer / follow-up / abort 的不同语义？
- ☐ Event stream 是否足以做日志、Eval、UI 与成本分析？
- ☐ World State、Agent State、Context State 是否分别管理？
- ☐ Subagent/Planner/MCP 是因为真实失败模式引入，还是因为“框架都这么做”？
- ☐ 重复成功的工作流是否有路径沉淀为 Skill？

Appendix E 参考资料

以下资料用于核对当前命令、目录、API 与设计定位。访问日期：2026-08-13。

- Pi Documentation: <https://pi.dev/docs/latest>
- Quickstart: <https://pi.dev/docs/latest/quickstart>
- Using Pi: <https://pi.dev/docs/latest/usage>
- Providers: <https://pi.dev/docs/latest/providers>
- Settings: <https://pi.dev/docs/latest/settings>
- Sessions: <https://pi.dev/docs/latest/sessions>
- Compaction: <https://pi.dev/docs/latest/compaction>
- Skills: <https://pi.dev/docs/latest/skills>
- Prompt Templates: <https://pi.dev/docs/latest/prompt-templates>
- Extensions: <https://pi.dev/docs/latest/extensions>
- Pi Packages: <https://pi.dev/docs/latest/packages>
- Custom Models: <https://pi.dev/docs/latest/models>
- Security: <https://pi.dev/docs/latest/security>
- Containerization: <https://pi.dev/docs/latest/containerization>
- SDK: <https://pi.dev/docs/latest/sdk>
- Repository: <https://github.com/earendil-works/pi>
- Original design discussion by Mario Zechner: <https://mariozechner.at/posts/2025-11-30-pi-coding-agent/>

版本策略 Pi 仍在快速迭代。任何涉及 Extension event 字段、SDK 类型、模型 Provider 兼容参数的代码，落地前应对照当前 Latest 文档与仓库类型定义。